

ARC RestEM Implementation

Overview

This directory contains the complete implementation of SEAL RestEM for ARC (Abstract Reasoning Corpus) tasks. The implementation follows the original SEAL methodology exactly - generating training configurations rather than direct solutions.

Core Files

Pipeline Components

- `arc_restem_pipeline.py` - Main orchestration of the 4-phase RestEM methodology
- `arc_self_edit_generator.py` - Phase 1: Generates training configurations (JSON objects)
- `arc_augmenters.py` - Sophisticated augmentation system (rotation, scaling, chaining, etc.)
- `arc_ttt_trainer.py` - Phase 2: Trains LoRA adapters using configurations
- `arc_solution_evaluator.py` - Phase 3: Evaluates adapter performance
- `arc_restem_trainer.py` - Phase 4: Behavioral cloning on successful configurations

Execution

- `run_arc_restem.py` - Main execution script with comprehensive logging

Quick Start

```
# From the restem directory
python3 run_arc_restem.py

# Or use the convenient shell script from SEAL root
cd /Users/keenana/Documents/SEAL
./knowledge-incorporation/scripts/run_arc_restem.sh
```

SEAL RestEM Methodology

This implementation correctly follows the original SEAL approach:

Phase 1: Configuration Generation

- Generates JSON training configurations (not solutions)
- Specifies augmentation strategies and training parameters
- Creates diverse training approaches for each ARC task

Phase 2: Test-Time Training (TTT)

- Trains LoRA adapters using configuration-specified augmentations
- Each configuration gets its own specialized adapter
- Uses MLX framework for efficient Apple Silicon training

Phase 3: Evaluation

- Tests each adapter on validation tasks
- Measures performance improvements
- Assigns binary rewards to successful configurations

Phase 4: Behavioral Cloning

- Trains final model on successful configurations only
- Uses supervised learning (not traditional RL)
- Creates model that generates effective training strategies

Key Features

SEAL Methodology Implementation

-  Configuration-based approach (generates training strategies, not solutions)
-  4-phase pipeline: Config generation → Adapter training → Evaluation → Behavioral cloning
-  Sophisticated augmentation system matching original SEAL exactly
-  MLX optimization for Apple Silicon

ARC Task Support

-  Loads real ARC data or creates mock tasks for testing
-  Handles visual pattern recognition with input/output grids
-  Supports both training and evaluation ARC datasets

Technical Excellence

-  Comprehensive error handling and logging
-  LoRA adapter training with gradient accumulation
-  Memory-efficient processing with cleanup
-  Configurable parameters for all phases

Expected Output

The pipeline generates:

1. **Training Configurations:** JSON objects specifying augmentation strategies
2. **LoRA Adapters:** Specialized models trained on augmented data
3. **Evaluation Results:** Performance scores for each configuration
4. **Behavioral Cloning Data:** Training data for final RestEM model

Output Structure

```
logs/arc_restem_experiments/  
├─ arc_restem_run_YYYYMMDD_HHMMSS.log      # Main execution log  
├─ arc_restem_YYYYMMDD_HHMMSS/  
│   └─ phase1_configs/  
│       └─ task1_configs.json              # Results directory  
                                           # Generated configurations
```

```

├── task2_configs.json
├── ...
├── phase2_adapters/                                # Trained LoRA adapters
│   ├── task1/
│   │   ├── adapter_0/
│   │   ├── adapter_1/
│   │   └── ...
│   └── task2/
├── phase3_evaluations/                            # Configuration
evaluations
│   ├── evaluation_results.json
│   ├── successful_configs.json
│   └── ...
├── phase4_restem/                                # Final behavioral cloning
│   └── behavioral_cloning_data.json
└── iteration_summary.json                        # Overall results

```

Configuration

Edit variables in `../scripts/run_arc_restem.sh`:

```

# Model Configuration
MODEL_NAME="mlx-community/Meta-Llama-3-8B-Instruct"

# Experiment Configuration
MAX_TASKS=3                # Number of ARC tasks to process
N_CONFIGS=15               # Number of configurations per task
RESTEM_EPOCHS=8            # Number of epochs for RestEM training

# Logging
LOG_LEVEL="INFO"           # DEBUG, INFO, WARNING, ERROR

```

Documentation

For complete usage instructions, see:

- [/Users/keenan/Documents/SEAL/ARC_RESTEM_COMPLETE_GUIDE.md](#)

Dependencies

```
pip install -r /Users/keenan/Documents/SEAL/requirements_mlx.txt
```

This implementation represents a faithful reproduction of the SEAL RestEM methodology applied to ARC reasoning tasks.